

DEVELOPER TRAINING LAB

Copilot + Skill + MCP CBS Troubleshooting Lab

Architecture, implementation, testing, debugging,
security, and an optional remediation handoff

User → Copilot → Skill → MCP Client → MCP Server
→ Application Service → CBS Domain Engine → Structured Result

Contents

1. Lab Overview
2. Incremental Development Plan
3. High-Level Architecture
4. Low-Level Design and Code Walkthrough
5. Complete Request and Protocol Flow
6. Skill and MCP Client Configuration
7. Testing and Debugging
8. Security and Observability
9. Optional Remediation Extension
10. Hands-On Exercises and Mental Model
11. Exact Source Map

Copilot + Skill + MCP CBS Troubleshooting Lab

This lab demonstrates a complete, working control flow:

```
User -> Copilot -> Skill -> SKILL.md -> MCP client -> MCP server
-> MCP tool -> application service -> CBS domain engine
-> structured result -> Copilot -> User
```

The implementation uses .NET 10 and the official `ModelContextProtocol` C# SDK. It analyzes local Windows Component Based Servicing (CBS) text logs without putting parsing or diagnosis logic in the MCP tool handlers.

Lab order

Do the lab in this order; do not start by reading the finished server:

Phase	Outcome	Material
1	Understand the HLD and boundaries	01-architecture.md
2	Understand classes, interfaces, and calls	02-llid-and-code.md
3	Build the solution and contracts	Source under <code>src\</code>
4	Start a minimal stdio MCP server	<code>Program.cs</code> , <code>CbsTools.cs</code>
5	Test MCP initialize, tools/list, tools/call	<code>tools\Cbs.Mcp.SmokeClient</code>
6	Configure the MCP client	04-skill-and-configuration.md
7	Add the CBS skill	<code>.github\skills\cbs-troubleshooter\SKILL.md</code>
8	Implement inspection and state	<code>CbsPathInspector</code> , <code>AnalysisRepository</code>
9	Implement streaming parsing	<code>CbsLogParser</code>
10	Implement session-aware diagnosis	<code>CbsDiagnosisEngine</code>
11	Add evidence and tracing	<code>CbsFailureTracer</code> , service methods
12	Add reports and persistence boundary	<code>CbsAnalysisArtifactWriter</code>
13	Run the complete pipeline	03-end-to-end-flow.md
14	Design remediation handoff	07-remediation-extension.md

Build and run

```
cd E:\copilot_mcp_skill_lab
dotnet build CbsCopilotLab.slnx
dotnet test CbsCopilotLab.slnx --no-build
dotnet run --project tools\Cbs.Mcp.SmokeClient --no-build -- `
  src\Cbs.Mcp.Server\bin\Debug\net10.0\Cbs.Mcp.Server.dll `
  samples\CBS
```

The smoke client launches the server over stdio, performs MCP initialization, lists the four tools, and calls `inspect_cbs`.

Deliverable map

Required deliverable	Location
Complete HLD and final HLD	docs\01-architecture.md
Incremental 14-phase build plan	docs\00-incremental-lab-plan.md
Complete LLD and class call diagrams	docs\02-lld-and-code.md
End-to-end workflow and sequence	docs\03-end-to-end-flow.md
Folder structure	docs\02-lld-and-code.md
Production-style skill	.github\skills\cbs-troubleshooter\SKILL.md
MCP configuration	.vscode\mcp.json , config\mcp.generic.json
MCP server and tools	src\Cbs.Mcp.Server
Contracts	src\Cbs.Mcp.Contracts
Parser and diagnosis engine	src\Cbs.Mcp.Server\Analysis
Testing and debugging	docs\05-testing-and-debugging.md
Security and observability	docs\06-security-and-observability.md
Remediation extension	docs\07-remediation-extension.md
Exercises, prompts, expected calls/results	docs\08-lab-exercises.md
Exact source map	docs\09-source-map.md

Scope

The core lab is read-only diagnosis. The remediation section is an architecture extension only: it produces a deterministic handoff contract but does not download or execute a remediation package.

0. Incremental Development Plan

Each phase has a concrete success gate. Complete the gate before moving to the next phase.

Phase 1 - Architecture and HLD

Build: Draw the user, Copilot, skill, MCP client/server, application, domain, infrastructure, and contract boundaries.

Learn: Ownership is more important than process names. Decide where parsing is forbidden before writing parsing code.

Success gate: A student can explain why `SKILL.md`, `CbsTools`, and `CbsDiagnosisEngine` are three different components.

Phase 2 - Create the solution and projects

```
dotnet new sln -n CbsCopilotLab
dotnet new classlib -n Cbs.Mcp.Contracts -o src\Cbs.Mcp.Contracts -f net10.0
dotnet new console -n Cbs.Mcp.Server -o src\Cbs.Mcp.Server -f net10.0
dotnet new xunit -n Cbs.Mcp.Tests -o tests\Cbs.Mcp.Tests -f net10.0
```

Success gate: All projects restore and appear in `CbsCopilotLab.slnx`.

Phase 3 - Define contracts

Create `ToolResult<T>`, inspection, parsed record, candidate, finding, diagnosis, evidence, and trace records.

Success gate: The contracts project builds without depending on the MCP server project.

Phase 4 - Create the minimal MCP server

Add `ModelContextProtocol` and `Microsoft.Extensions.Hosting`. Configure Generic Host, stderr logging, stdio transport, and assembly tool discovery.

Success gate: The process starts and waits for MCP input without printing diagnostics to stdout.

Phase 5 - Expose a test tool

Create one attributed tool and connect with an MCP client.

Success gate: `initialize`, `tools/list`, and one `tools/call` complete.

Phase 6 - Configure the Copilot MCP client

Add the product-appropriate server entry. For VS Code, use `.vscode\mcp.json`; for another host, translate the same command/args/transport into its supported configuration.

Success gate: The Copilot host lists the server's tools.

Phase 7 - Create `SKILL.md`

Add the trigger description, inspect-first workflow, evidence/trace escalation, result presentation, and safety rules.

Success gate: A CBS prompt activates the skill; an unrelated log prompt does not.

Phase 8 - Implement `inspect_cbs`

Build path validation, safe enumeration, classification, analysis ID creation, workspace creation, and repository insertion.

Success gate: A valid directory returns an ID; invalid, missing, and inaccessible paths return structured errors.

Phase 9 - Implement the parser

Use `FileStream` and `StreamReader`, preserve line numbers and exact text, and extract timestamp, severity, session, package, HRESULT, and terminal state.

Success gate: Known CBS lines produce expected records and candidates without `File.ReadAllLines`.

Phase 10 - Implement diagnosis

Group by session, determine terminal states, compute last success, suppress superseded failures, deduplicate, rank, and map findings to recommendations.

Success gate: An error followed by terminal success is not reported as the root cause.

Phase 11 - Add evidence and tracing

Expose exact finding evidence and causal stage tracing.

Success gate: Evidence contains file and line; trace stays within the selected finding's session.

Phase 12 - Add state and artifacts

Add repository expiry, per-analysis locks, cached diagnosis, and JSON/text reports.

Success gate: Repeated diagnosis returns stable cached state and concurrent calls do not duplicate work.

Phase 13 - Run the complete Copilot-to-MCP test

Use the sample CBS directory, observe inspect then diagnose, and request evidence and a trace.

Success gate: The result suppresses the historical successful session and identifies the current failed session.

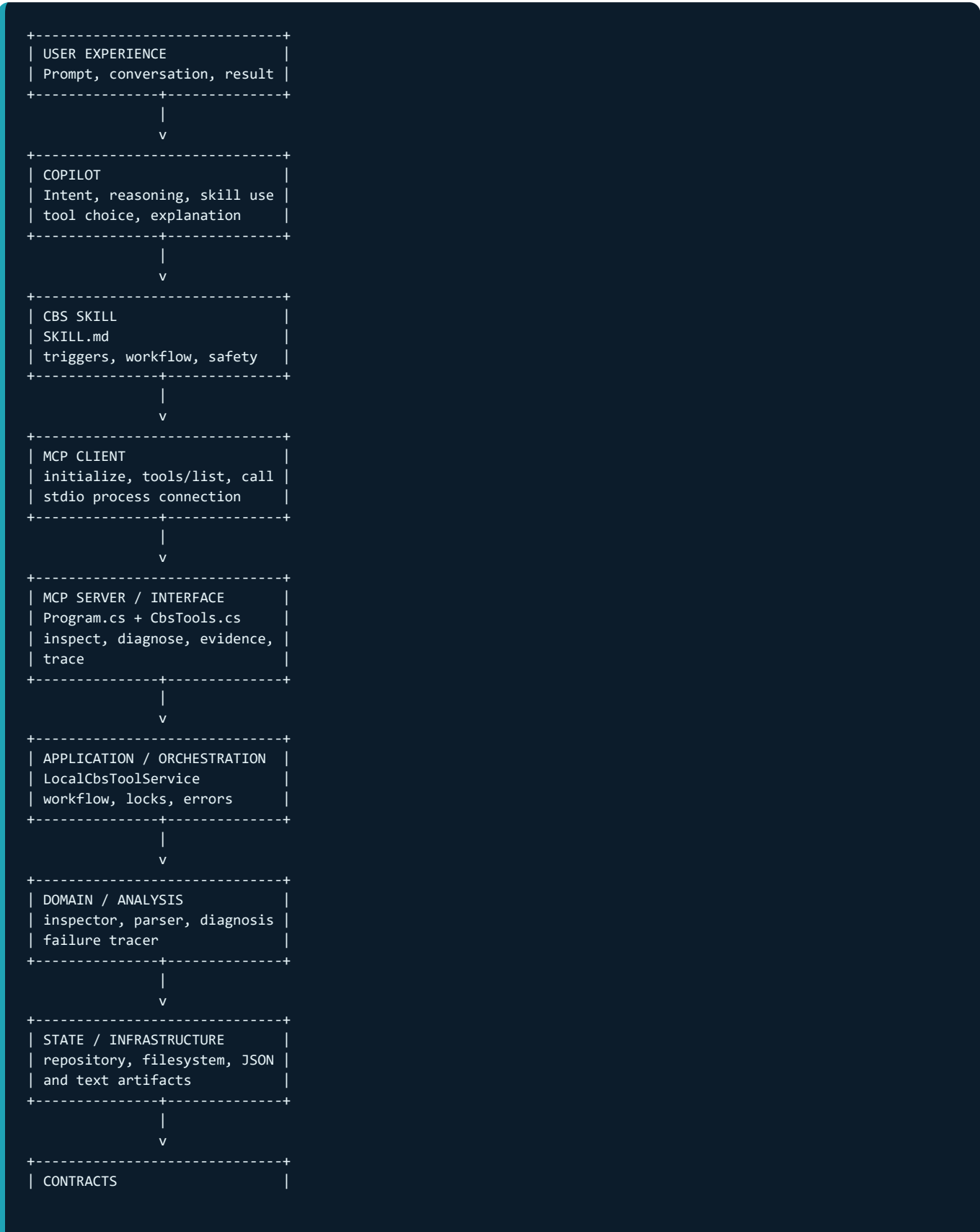
Phase 14 - Add remediation handoff

Define a deterministic `actionId` handoff without implementing arbitrary command execution.

Success gate: Diagnosis remains read-only; remediation requires a separate consented and authorized boundary.

1. High-Level Architecture

1.1 Big picture



1.2 Why each box exists

Layer 1 - User experience

Why: A person needs a natural-language entry point and a comprehensible diagnosis.

Owns: The prompt, clarification, consent for later remediation, and display of the answer.

Does not own: File enumeration, CBS parsing, HRESULT correlation, or direct remediation.

Control transfer: The prompt is sent to Copilot. The final structured tool result is translated back into a user-facing explanation.

Test: Ask a representative prompt and verify that the answer includes the current failure, evidence, uncertainty, and next action without fabricating facts.

Layer 2 - Copilot

Why: The solution needs an intelligence and orchestration layer that can understand intent and choose capabilities.

Owns: Intent recognition, skill discovery, following skill instructions, MCP tool selection, carrying `analysisId` between calls, and explaining results.

Lower-level implementation: The Copilot host contains an MCP client. The repository contributes a skill and MCP configuration; it does not implement Copilot itself.

Control transfer: Copilot matches the request to the CBS skill, reads its workflow, and asks the MCP client to call `inspect_cbs`.

Error behavior: Copilot should present structured errors and follow their next-step guidance. It must not replace a failed tool call with a guessed diagnosis.

Test: Verify that a CBS prompt activates the skill and that `inspect_cbs` precedes `diagnose_cbs`.

Layer 3 - Skill

Why: A general model needs reusable, domain-specific operating instructions.

Owns: Trigger conditions, the inspect-before-diagnose workflow, evidence and trace escalation, safety rules, and output interpretation.

Artifact: `.github\skills\cbs-troubleshooter\SKILL.md`.

Does not own: Parsing code or MCP transport.

Control transfer: The skill tells Copilot which named tool to call next and how to interpret its contract.

Test: Give prompts for CBS, unrelated logs, proof requests, and causal trace requests; inspect which skill and tool sequence is selected.

Layer 4 - MCP client

Why: Copilot needs a protocol implementation for discovering and invoking external capabilities.

Owns: Process launch, MCP initialization, capability negotiation, `tools/list`, `tools/call`, cancellation, and serialization.

Lower-level implementation: The Copilot product supplies this layer. `Cbs.Mcp.SmokeClient` is the lab's transparent reference client.

Control transfer: It serializes tool name and arguments into MCP JSON-RPC messages over stdio and receives content blocks.

Test: Launch the server, list tools, and call `inspect_cbs`.

Layer 5 - MCP server/interface

Why: The CBS capability needs a stable, discoverable external boundary.

Owns: Server startup, stdio transport, tool metadata, parameter binding, and returning serializable results.

Implemented by: `Program.cs` and `Tools\CbsTools.cs`.

Control transfer: `CbsTools.InspectCbsAsync` forwards to `ILocalCbsToolService.InspectAsync`.

Design rule: Keep handlers thin. Tool descriptions guide model selection; handlers should not become the business layer.

Test: Verify all four names appear in `tools/list`, descriptions are precise, invalid parameters return structured errors, and stdout contains protocol traffic only.

Layer 6 - Application/orchestration

Why: A use case crosses several domain and infrastructure components and needs one transaction-like coordinator.

Owns: Analysis lookup, locking, caching, call ordering, artifact generation, logging, and exception-to-contract translation.

Implemented by: `ILocalCbsToolService` and `LocalCbsToolService`.

Control transfer: It calls path inspection, parser, diagnosis engine, tracer, repository, and artifact writer as required by each tool.

Test: Mock or use real collaborators to verify successful workflows, missing analysis IDs, cancellation, and cached diagnosis reuse.

Layer 7 - Domain/analysis

Why: CBS intelligence must remain reusable and testable without MCP.

Owns: Safe artifact classification, streaming parsing, session correlation, HRESULT extraction, successful-session suppression, ranking, and causal trace construction.

Implemented by: `CbsPathInspector`, `CbsLogParser`, `CbsDiagnosisEngine`, and `CbsFailureTracer`.

Control transfer: Domain methods accept contracts or domain records and return deterministic data. They do not know about JSON-RPC or Copilot.

Test: Use known log lines and session timelines. Especially verify that a later successful terminal state suppresses an earlier intermediate error.

Layer 8 - State/infrastructure

Why: One conversation uses several independent MCP calls, and reports must survive beyond a single method stack.

Owns: In-memory analysis state, expiry, per-analysis locks, local workspaces, and JSON/text reports.

Implemented by: `AnalysisRepository`, `AnalysisState`, `CbsAnalysisArtifactWriter`, and the local filesystem.

Control transfer: `inspect_cbs` stores state keyed by `analysisId`; later tools retrieve and enrich it.

```
inspect_cbs
-> A-123 stored
-> diagnose_cbs(A-123)
-> show_cbs_evidence(A-123)
-> trace_cbs_failure(A-123)
```

Test: Verify lookup, expiry behavior, concurrent diagnosis serialization, cache reuse, and report creation.

Contracts

Why: Every boundary needs explicit, serializable, implementation-independent meaning.

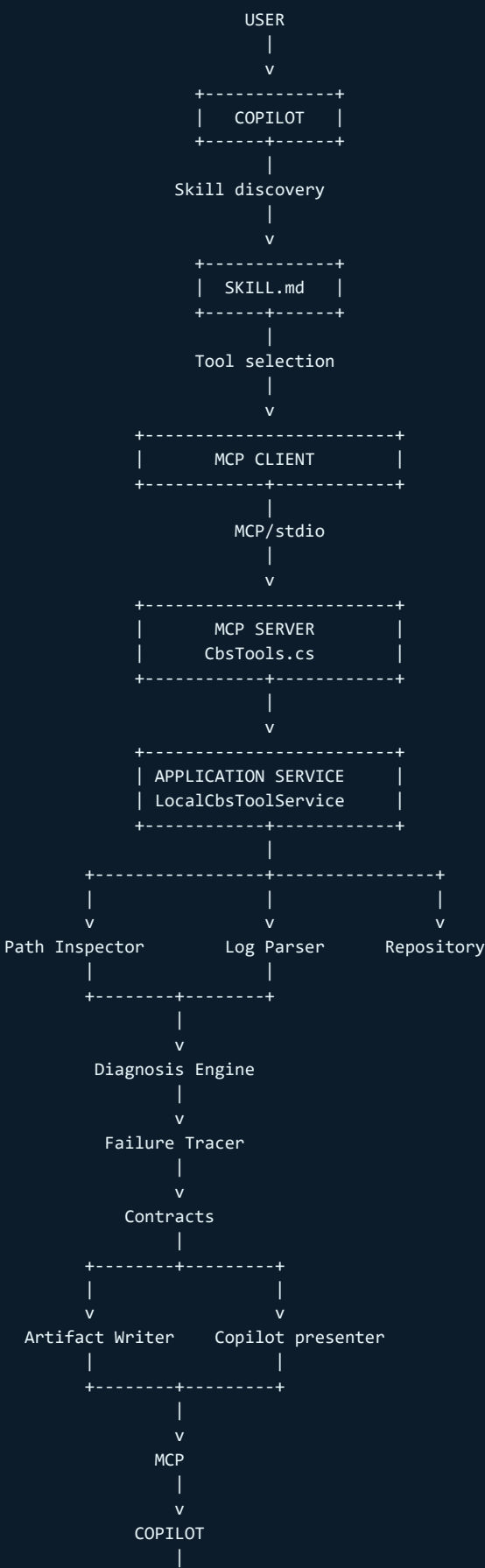
Owns: Input/output shape, optionality, enum values, evidence references, and error envelopes.

Implemented by: The `Cbs.Mcp.Contracts` project.

Control transfer: Contracts pass through domain, application, MCP serialization, and Copilot interpretation.

Test: Serialize representative success/failure objects and reject semantically invalid combinations in contract tests or consumers.

1.3 Final polished HLD



1.4 Component comparison

Component	Purpose	Key boundary
Copilot	Reasoning and orchestration	Natural language to workflow
Skill	Reusable instructions	Intent to ordered tool use
SKILL.md	Skill definition artifact	Declarative guidance, not code
MCP client	Protocol consumer	Host to server JSON-RPC
MCP server	Capability provider	External protocol endpoint
MCP tool	Individual operation	Named typed request/response
Application service	Use-case orchestration	Tool call to collaborators
Domain engine	CBS intelligence	Records to diagnosis
Contract	Structured information	Stable data across boundaries

SKILL.md != MCP server. The skill tells Copilot **what workflow to follow**. MCP provides **how an external operation is discovered and executed**.

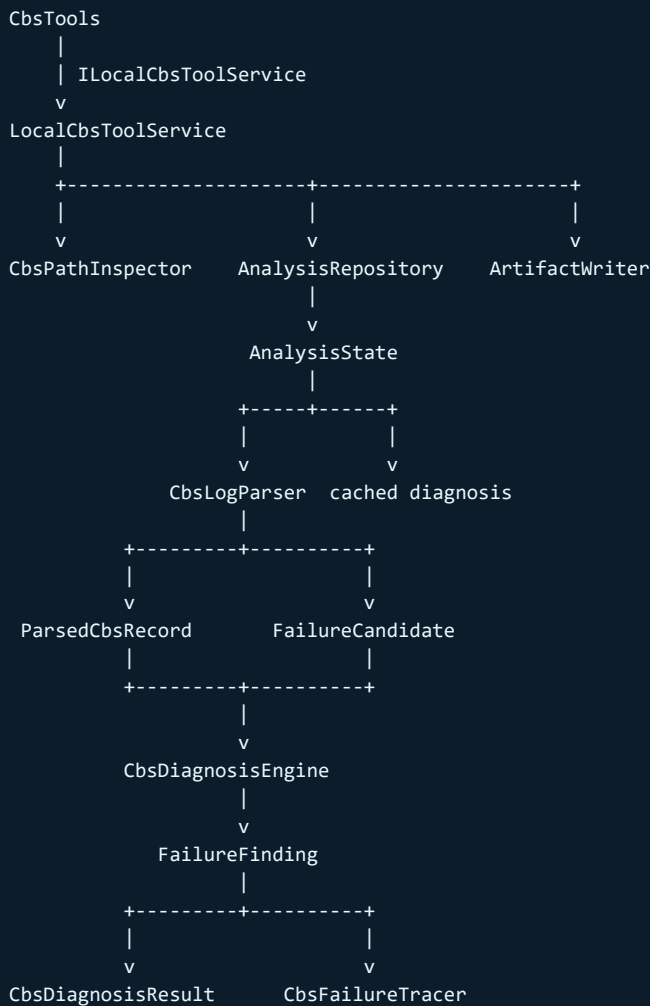
2. Low-Level Design and Code Walkthrough

2.1 Folder structure

```
E:\copilot_mcp_skill_lab
|-- .github\skills\cbs-troubleshooter\SKILL.md
|-- .vscode\mcp.json
|-- config\mcp.generic.json
|-- docs\
|-- samples\CBS\CBS.log
|-- src\
|   |-- Cbs.Mcp.Contracts\
|   |   |-- ToolResult.cs
|   |   |-- CbsArtifact.cs
|   |   |-- CbsInspectionResult.cs
|   |   |-- ParsedCbsRecord.cs
|   |   |-- FailureFinding.cs
|   |   |-- CbsDiagnosisResult.cs
|   |   |-- RemediationHandoff.cs
|   |-- Cbs.Mcp.Server\
|   |   |-- Program.cs
|   |   |-- Tools\CbsTools.cs
|   |   |-- Services\ILocalCbsToolService.cs
|   |   |-- Services\LocalCbsToolService.cs
|   |   |-- Analysis\CbsPathInspector.cs
|   |   |-- Analysis\CbsLogParser.cs
|   |   |-- Analysis\CbsDiagnosisEngine.cs
|   |   |-- Analysis\CbsFailureTracer.cs
|   |   |-- State\AnalysisState.cs
|   |   |-- State\AnalysisRepository.cs
|   |   |-- Infrastructure\CbsAnalysisArtifactWriter.cs
|-- tests\Cbs.Mcp.Tests\
|-- tools\Cbs.Mcp.SmokeClient\
```

Folders express architectural boundaries. `Contracts` has no server dependency. `Tools` knows only the application interface. `Analysis` knows CBS rules but not MCP. `State` and `Infrastructure` isolate mutable state and I/O.

2.2 Class relationship diagram



2.3 Program.cs

Purpose: Build the process, dependency graph, MCP server, and stdio transport.

```
var builder = Host.CreateApplicationBuilder(args);
builder.Logging.AddConsole(options =>
{
    options.LogToStandardErrorThreshold = LogLevel.Trace;
});

builder.Services.AddSingleton<CbsPathInspector>();
builder.Services.AddSingleton<ILocalCbsToolService, LocalCbsToolService>();

builder.Services
    .AddMcpServer()
    .WithStdioServerTransport()
    .WithToolsFromAssembly();

await builder.Build().RunAsync();
```

Syntax:

- `var builder` : local variable with compile-time inferred type.
- `Host.CreateApplicationBuilder(args)` : creates a .NET Generic Host builder.
- `AddSingleton<T>()` : registers one dependency-injection instance for the process lifetime.
- `AddMcpServer()` : registers MCP server services.
- `WithStdioServerTransport()` : binds MCP protocol input/output to stdin/stdout.
- `WithToolsFromAssembly()` : discovers `[McpServerToolType]` classes and `[McpServerTool]` methods.
- `await ... RunAsync()` : starts the asynchronous host until shutdown.

Runtime flow: Process starts -> DI graph is built -> MCP reads stdin -> tool requests are dispatched -> shutdown cancellation stops the host.

HLD layer: MCP server bootstrap plus infrastructure composition root.

Critical rule: stdout is the stdio protocol channel. Console logging is configured to stderr; arbitrary `Console.WriteLine` in the server can corrupt JSON-RPC framing.

Test: The smoke client proves initialization and tool discovery.

2.4 CbsTools

Responsibility: Publish tool names, descriptions, parameter descriptions, and forward calls.

Constructor dependency: `ILocalCbsToolService service` .

```
public Task<ToolResult<CbsInspectionResult>> InspectCbsAsync(
    string? path,
    CancellationToken cancellationToken) =>
    service.InspectAsync(path, cancellationToken);
```

Syntax:

- `public` : callable by the MCP SDK after discovery.
- `Task<T>` : asynchronous operation whose eventual value is `T` .
- `ToolResult<CbsInspectionResult>` : a generic envelope carrying either typed data or a typed error.
- `string?` : nullable reference; invalid/missing values are handled by the application workflow.
- `CancellationToken` : cooperative cancellation propagated from client to filesystem work.
- `=>` : expression-bodied method; it performs one forwarding call.

Caller: MCP SDK tool dispatcher.

Callee: `ILocalCbsToolService.InspectAsync` .

Input: `{ "path": "C:\\Logs\\CBS" }` .

Output: `ToolResult<CbsInspectionResult>` .

Why no parsing here: Parsing would couple protocol metadata to domain rules, make handlers hard to test, duplicate workflows, and prevent reuse outside MCP.

Tool methods

Method	MCP name	Input	Output	Next call
InspectCbsAsync	inspect_cbs	path	inspection + analysis ID	DiagnoseCbsAsync
DiagnoseCbsAsync	diagnose_cbs	analysis ID	ranked diagnosis	evidence or trace
ShowCbsEvidenceAsync	show_cbs_evidence	analysis ID, optional finding ID	exact lines	Copilot presentation
TraceCbsFailureAsync	trace_cbs_failure	analysis ID, optional finding ID	ordered causal stages	Copilot explanation

Tool descriptions are operational metadata for the model. “Always call this before diagnose” and “requires analysisId” reduce invalid tool ordering.

2.5 ILocalCbsToolService

Purpose: Keep MCP tools dependent on a stable application contract rather than a concrete implementation.

```
Task<ToolResult<CbsDiagnosisResult>> DiagnoseAsync(
    string analysisId,
    CancellationToken cancellationToken);
```

Input contract: A non-empty analysis ID created by inspection.

Output contract: Success with a diagnosis or failure with `ToolError`.

Called by: `CbsTools`.

Implemented by: `LocalCbsToolService`.

Test: Substitute a fake implementation when testing tool forwarding.

2.6 LocalCbsToolService

Responsibility: Coordinate use cases and translate internal failures at the MCP boundary.

Constructor dependencies:

Dependency	Reason
CbsPathInspector	Validate and inventory input
CbsLogParser	Stream and structure evidence

Dependency	Reason
CbsDiagnosisEngine	Correlate and rank failures
CbsFailureTracer	Build a causal sequence
AnalysisRepository	Preserve multi-call state
CbsAnalysisArtifactWriter	Persist reports
ILogger	Diagnostics on stderr

InspectAsync line-by-line logic

```
var (inspection, artifacts) =
    await inspector.InspectAsync(path, cancellationToken);
```

The inspector validates the root, safely enumerates it, creates an analysis ID, and returns two related values:

- `inspection` : public summary sent to the client.
- `artifacts` : complete internal inventory stored for later parsing.

Returning both prevents later tools from re-enumerating a directory that may change between calls.

```
var workspace = artifactWriter.CreateWorkspace(inspection.AnalysisId);
inspection = inspection with { WorkspaceDirectory = workspace };
```

The first line creates a deterministic per-analysis output directory. The second uses record `with` syntax to create a copy with the workspace populated; records remain immutable at contract boundaries.

```
repository.Add(inspection, artifacts);
```

This is required because `diagnose_cbs` is a separate MCP request. The original method stack will no longer exist when that request arrives.

```
return ToolResult<CbsInspectionResult>.FromData(inspection);
```

The application returns a structured envelope rather than throwing across MCP. Copilot can reliably distinguish success from `INVALID_PATH`, `ACCESS_DENIED`, or `IO_ERROR`.

Inspect call chain

```
CbsTools.InspectCbsAsync()  
-> ILocalCbsToolService.InspectAsync()  
-> LocalCbsToolService.InspectAsync()  
-> CbsPathInspector.InspectAsync()  
-> filesystem enumeration  
-> CbsAnalysisArtifactWriter.CreateWorkspace()  
-> AnalysisRepository.Add()  
-> ToolResult<CbsInspectionResult>
```

Diagnosis call chain

```
CbsTools.DiagnoseCbsAsync()  
-> LocalCbsToolService.DiagnoseAsync()  
-> AnalysisRepository.TryGet()  
-> AnalysisState.Gate.WaitAsync()  
-> CbsLogParser.ParseAsync()  
-> CbsDiagnosisEngine.Diagnose()  
-> CbsAnalysisArtifactWriter.WriteDiagnosisAsync()  
-> cache AnalysisState.Diagnosis  
-> ToolResult<CbsDiagnosisResult>
```

The per-analysis `SemaphoreSlim` prevents two simultaneous calls from parsing and writing the same analysis concurrently.

2.7 CbsPathInspector

Responsibility: Convert an untrusted path into a bounded CBS artifact inventory.

Algorithm:

```
Validate non-empty path  
-> expand environment variables  
-> canonicalize with Path.GetFullPath  
-> require existing directory  
-> reject reparse-point root  
-> walk directories without following child reparse points  
-> classify CBS.log, CbsPersist*, Sessions.xml  
-> enforce file-count and aggregate-size limits  
-> mark text artifacts as parseable  
-> create analysis ID  
-> return inspection + inventory
```

Input: Nullable user path and cancellation token.

Output: Tuple of `CbsInspectionResult` and `IReadOnlyList<CbsArtifact>`.

Errors: Argument, missing directory, unauthorized access, I/O, safety-limit violations, or cancellation. The application layer maps these to tool errors.

Test: Existing CBS directory, missing directory, reparse point, artifact limit, and cancellation.

2.8 CbsLogParser

Responsibility: Stream parseable artifacts into exact source records and failure candidates.

```
while (await reader.ReadLineAsync(cancellationToken) is { } line)
{
    lineNumber++;
    if (!LooksRelevant(line))
    {
        continue;
    }

    yield return ParseLine(path, lineNumber, line);
}
```

Why streaming: `File.ReadAllLines` allocates an array and every line for the complete file. CBS logs can be large. `FileStream` plus `StreamReader` reads sequentially, honors cancellation, tolerates concurrent CBS writers through `FileShare.ReadWrite` | `FileShare.Delete`, and retains bounded working memory.

Produced fields: Source path, line number, verbatim evidence, timestamp, severity, component, session, package, HRESULT, message, and terminal state.

Evidence invariant: `Evidence` is the original line and `LineNumber` is captured before parsing. Diagnosis may summarize, but proof remains traceable.

FailureCandidate: A projection of records with errors, HRESULTs, failure terminal states, rollback, or failure language. It carries a base score, not a final diagnosis.

Test: Known formats, missing optional fields, lowercase HRESULT, malformed timestamp, huge streamed file, cancellation, and file sharing.

2.9 CbsDiagnosisEngine

Responsibility: Turn noisy candidates into current ranked findings.

```
Parsed records
-> group by servicing session
-> locate each session's terminal record
-> find last successful servicing point
-> locate latest completed session
-> suppress candidate if its own session later succeeded
-> suppress historical candidate before a later success
-> deduplicate by HRESULT/session/normalized message
-> rank by terminal state, HRESULT, repetition, and context
-> create FailureFinding
-> build CbsDiagnosisResult
```

The central rule

```
Session 100:
  10:00 Error 0x800F081F
  10:05 terminal success
  => suppress the error

Session 101:
  11:00 Error 0x800F081F
  11:04 terminal failure
  => retain and rank the error
```

An `Error` line is evidence, not automatically a root cause. Session outcome can change its meaning.

Input: Analysis ID, parsed records, and candidates.

Output: Immutable `CbsDiagnosisResult`.

Error handling: The pure engine does not catch programming errors. The application service owns boundary translation.

Test: Successful-session suppression, historical suppression, latest failed session, deduplication, ranking, and no-finding result.

2.10 `CbsFailureTracer`

Responsibility: Explain temporal causality rather than only ranking the best cause.

```
Initiation
-> PrimaryFailure
-> DerivedFailure
-> Consequence
-> Rollback
-> TerminalResult
```

Diagnosis asks, "What is the best current root-cause finding?" Tracing asks, "What ordered chain led from servicing start to terminal outcome?"

The tracer filters records by the selected finding's session or HRESULT, orders by timestamp and source line, selects representative stages, and returns exact evidence for each stage.

Test: Complete chain, absent optional stages, rollback-only sequences, same-timestamp line ordering, and unknown finding ID.

2.11 State and infrastructure

`AnalysisState`

Holds immutable inspection/artifacts and mutable cached records, candidates, and diagnosis. `Gate` serializes work for one analysis. `LastAccessedAt` supports expiry.

AnalysisRepository

Uses `ConcurrentDictionary<string, AnalysisState>` for thread-safe lookup. Entries expire after two hours. This lab uses process-local memory: a server restart invalidates IDs.

For production, replace this implementation behind the same conceptual boundary with encrypted durable storage or a distributed cache if server restarts and horizontal scaling must preserve analyses.

CbsAnalysisArtifactWriter

Creates `%LOCALAPPDATA%\CbsMcpLab\analyses\<analysisId>` and writes `diagnosis.json` plus `diagnosis.txt`. It receives a completed diagnosis; it does not decide root cause.

2.12 Contracts

Contract	Meaning
<code>ToolResult<T></code>	Success/data or failure/error envelope
<code>ToolError</code>	Stable code, safe message, retryability, details
<code>CbsArtifact</code>	Inspected file metadata
<code>CbsInspectionResult</code>	Public inventory and analysis handle
<code>ParsedCbsRecord</code>	Structured exact source record
<code>FailureCandidate</code>	Unresolved potential failure
<code>FailureFinding</code>	Ranked, explained diagnosis
<code>CbsDiagnosisResult</code>	Analysis-level result
<code>EvidenceReference</code>	File, line, exact text
<code>CbsFailureTraceResult</code>	Ordered causal sequence
<code>RemediationHandoff</code>	Deterministic diagnosis-to-remediation intent

Contracts are in a separate project so the domain, server, tests, clients, and future remediation adapter can share data without taking a dependency on MCP hosting.

2.13 Final LLD

```
CbsTools.InspectCbsAsync()
  |
  v
ILocalCbsToolService.InspectAsync()
  |
  v
LocalCbsToolService.InspectAsync()
  |
  +-----+
  |               |
  v               v
CbsPathInspector.InspectAsync()  ArtifactWriter.CreateWorkspace()
  |
  v
AnalysisRepository.Add()
  |
  v
ToolResult<CbsInspectionResult>
```

```
CbsTools.DiagnoseCbsAsync()
  |
  v
LocalCbsToolService.DiagnoseAsync()
  |
  v
AnalysisRepository.TryGet()
  |
  v
CbsLogParser.ParseAsync()
  |
  +-----+
  |               |
  v               v
ParsedCbsRecord[]  FailureCandidate[]
  |               |
  +-----+
  |
  v
CbsDiagnosisEngine.Diagnose()
  |
  v
CbsDiagnosisResult
  |
  +-----+
  |               |
  v               v
ArtifactWriter.Write()  AnalysisState cache
```

3. Complete Request and Protocol Flow

3.1 User request

Analyze C:\Logs\CBS and tell me why the update failed.

3.2 End-to-end workflow

```
User
-> Copilot recognizes CBS intent
-> CBS skill is activated
-> SKILL.md requires inspection
-> MCP client calls inspect_cbs
-> CbsTools forwards to application service
-> path inspector inventories evidence
-> repository stores analysis state
-> structured inspection returns analysisId
-> skill requires diagnosis
-> MCP client calls diagnose_cbs
-> parser streams logs
-> engine correlates sessions and terminal states
-> artifact writer persists reports
-> structured diagnosis returns
-> Copilot explains the current failure
```

3.3 Numbered trace with code ownership

1. **Copilot receives the prompt.** The user-experience layer supplies natural language; no repository code runs yet.
2. **Copilot recognizes CBS troubleshooting intent.** The words CBS, Windows Update failure, servicing, package, or servicing HRESULT match the skill description.
3. **Copilot loads the CBS skill.** The artifact is `.github\skills\cbs-troubleshooter\SKILL.md`.
4. `SKILL.md` **says inspect first.** This prevents diagnose calls without stable state.
5. **Copilot requests** `inspect_cbs`.

```
{
  "path": "C:\\Logs\\CBS"
}
```

6. **The MCP client sends a** `tools/call` **JSON-RPC request over stdio.** Framing and serialization are owned by the MCP SDK.

7. `CbsTools.InspectCbsAsync` **receives bound arguments**. The SDK uses the tool attribute and method signature.
8. **The tool forwards to** `ILocalCbsToolService.InspectAsync` .
9. `LocalCbsToolService.InspectAsync` **calls** `CbsPathInspector.InspectAsync` .
10. **The inspector validates and inventories files**. It canonicalizes the path, avoids reparse-point traversal, enforces bounds, and classifies artifacts.
11. **The application creates a workspace and calls** `AnalysisRepository.Add` .
12. **Inspection returns an analysis handle**.

```
{
  "success": true,
  "data": {
    "analysisId": "A-123",
    "servicingEvidenceFound": true,
    "artifacts": [
      {
        "path": "C:\\Logs\\CBS\\CBS.log",
        "kind": "CbsTextLog",
        "parseable": true
      }
    ]
  },
  "error": null
}
```

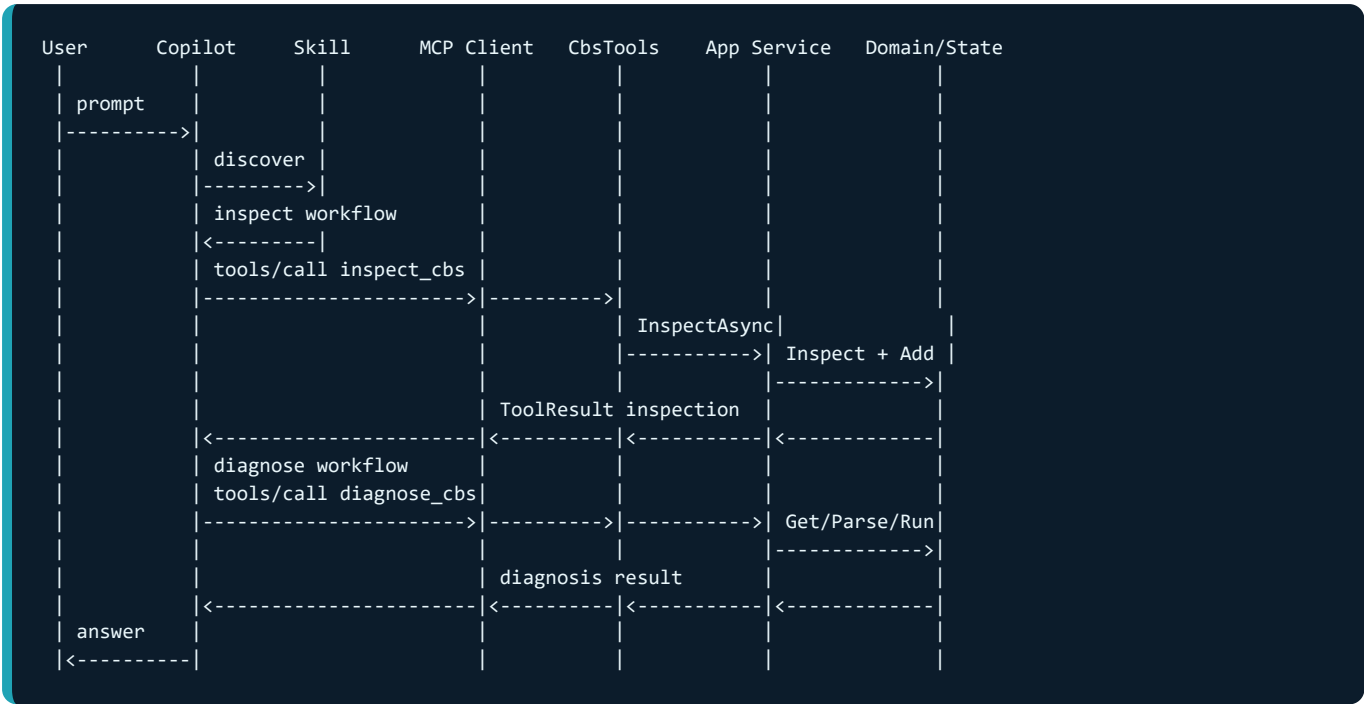
13. **Copilot continues the skill workflow**. Because servicing evidence exists, diagnosis is next.
14. **Copilot calls** `diagnose_cbs` .

```
{
  "analysisId": "A-123"
}
```

15. `LocalCbsToolService` **retrieves** `AnalysisState` . A missing or expired ID becomes `ANALYSIS_NOT_FOUND` .
16. `CbsLogParser.ParseAsync` **streams the inventoried text logs**. It retains source file and line numbers.
17. `CbsDiagnosisEngine.Diagnose` **correlates sessions, terminal states, failures, and HRESULT families**.
18. **The engine creates** `FailureFinding` **objects**. Historical or intermediate errors are suppressed before ranking.
19. `CbsAnalysisArtifactWriter` **writes JSON and text reports**.
20. **MCP returns** `ToolResult<CbsDiagnosisResult>` .
21. **Copilot interprets the contract according to the skill**. It preserves HRESULTS and distinguishes current from historical evidence.
22. **Copilot presents a user-friendly diagnosis**.

The latest failed servicing session is 101_1. The highest-ranked current failure is 0x800F081F for Package_for_KB5000002. An earlier occurrence in session 100_1 was suppressed because that session later completed successfully. The recommended action is to provide a matching repair source and repair the component store before retrying the update.

3.4 Sequence diagram



3.5 MCP protocol view



JSON-RPC is the request/response envelope used by MCP. Conceptually, each request has a method, an ID for correlation, and parameters; the response carries the matching ID and either a result or protocol-level error. Tool-level domain errors remain inside `ToolResult<T>` so expected conditions such as a missing path are machine-readable and do not masquerade as transport failures.

The MCP SDK owns exact wire serialization. Application code should not manually print JSON-RPC.

3.6 Evidence escalation

If the user asks, "Show me the proof," Copilot calls:

```
{  
  "analysisId": "A-123",  
  "findingId": "F-456"  
}
```

`show_cbs_evidence` returns exact `sourcePath`, `lineNumber`, and `text`.

If the user asks, "Trace how it failed and rolled back," Copilot calls `trace_cbs_failure` with the same identifiers. The returned stages are a temporal explanation, not a new diagnosis.

4. Skill and MCP Client Configuration

4.1 Why SKILL.md exists

The model can see tools, but tool availability alone does not encode the safest multi-step operating procedure. The skill adds domain workflow:

```
CBS intent
-> inspect first
-> require analysisId
-> diagnose only when evidence exists
-> request exact evidence when needed
-> request a causal trace when needed
-> preserve HRESULTS and avoid invented causes
```

The implementation lives in C#. The workflow lives in SKILL.md. Changing the wording or ordering guidance does not require changing the parser.

4.2 Skill sections

Section	Why it exists
YAML name	Stable skill identity
YAML description	Discovery trigger; it must mention concrete user intents
Objective	Declares the MCP server as the evidence source
Required workflow	Orders the tools and defines stop conditions
Presentation rules	Shapes the final user answer
Safety rules	Prevents invented evidence and accidental remediation
Tool selection table	Makes intent-to-tool mapping explicit
Error interpretation	Converts machine error codes into next actions

The skill is stored at .github\skills\cbs-troubleshooter\SKILL.md for project-level discovery. In GitHub Copilot CLI, /skills is the management surface. Product support and skill discovery locations can differ, so verify the host's current documentation.

4.3 Generic stdio configuration

```
{
  "servers": {
    "cbs": {
      "type": "stdio",
      "command": "dotnet",
      "args": [
        "E:\\copilot_mcp_skill_lab\\src\\Cbs.Mcp.Server\\bin\\Debug\\net10.0\\Cbs.Mcp.Server.dll"
      ],
      "cwd": "E:\\copilot_mcp_skill_lab",
      "env": {
        "DOTNET_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Field	Meaning
cbs	Client-local server name; not an MCP tool name
type	Transport. <code>stdio</code> launches a child process and connects its standard streams
command	Executable to start; here the .NET host
args	Arguments to the command; here the built server DLL
cwd	Predictable working directory for relative paths and process behavior
env	Non-secret process settings. Use the host's secret-input mechanism for secrets

4.4 Product-specific separation

Do not copy one configuration shape into every Copilot product.

Host	How to approach configuration
GitHub Copilot CLI	Use <code>/mcp</code> to manage MCP server configuration and <code>/skills</code> to inspect skills. Confirm the exact persisted config with the installed CLI version.
VS Code with GitHub Copilot	Use workspace <code>.vscode\mcp.json</code> or the MCP configuration UI. This lab includes <code>.vscode\mcp.json</code> .
Visual Studio with GitHub Copilot	Use Visual Studio's MCP server support and its supported configuration surface; do not assume VS Code paths.
Microsoft Copilot products	Capabilities differ by product. Confirm whether the product supports direct MCP, connectors, plugins, or an agent gateway. A local stdio executable may not be usable by a cloud-hosted product.
Copilot Studio	Treat it as a managed/cloud agent platform. Expose a supported remote capability or connector if direct local stdio is unavailable. Apply authentication and consent.

Host	How to approach configuration
Custom Copilot host	Use an MCP SDK client directly. <code>Cbs.Mcp.SmokeClient</code> demonstrates <code>StdioClientTransport</code> , <code>McpClient.CreateAsync</code> , <code>ListToolsAsync</code> , and <code>CallToolAsync</code> .

The invariant is architectural, not product-specific:

```
Host reasoning layer
-> MCP client
-> configured transport
-> CBS MCP server
```

4.5 How descriptions influence tool selection

Compare weak and strong descriptions:

```
Weak: "Inspects CBS."

Strong: "Inspect a local directory for Windows CBS servicing evidence.
Always call this before diagnose_cbs. Returns an analysisId for later calls."
```

The strong description supplies:

- The applicable domain
- The expected input
- Ordering constraint
- Output dependency
- Relationship to another tool

Descriptions are not security controls. The application service still validates every input and analysis ID.

4.6 Skill test prompts

Prompt	Expected behavior
"Analyze <code>C:\Logs\CBS</code> and find why update failed."	Skill activates; inspect then diagnose
"Show exact lines proving that CBS diagnosis."	Evidence tool after diagnosis
"Trace the rollback chain."	Failure trace tool
"Analyze an IIS access log."	CBS skill should not activate
"Run DISM to fix it."	Core skill should separate diagnosis from remediation and request consent in a future remediation flow

5. Testing and Debugging

5.1 Test pyramid by layer

Layer	Test	Expected assertion
Contracts	Serialize success/error DTOs	Stable property names and nullability
Path inspection	Inspect controlled directory	Correct classification, limits, and warnings
Parser	Parse known CBS lines	Timestamp/session/package/HRESULT/evidence retained
Diagnosis	Session later succeeds	Earlier candidate is suppressed
Diagnosis	Latest session terminal failure	Current candidate survives and ranks first
Failure trace	Ordered session records	Correct optional stages and exact evidence
Application service	Missing analysis ID	ANALYSIS_NOT_FOUND , no exception crosses boundary
MCP connectivity	Initialize and list tools	Four expected tools appear
MCP call	Call inspect_cbs	Structured success and analysis ID
Skill	CBS prompt	Correct skill and inspect-before-diagnose order
End to end	Prompt through explanation	No invented facts; current/historical distinction preserved

5.2 Commands

```
dotnet build CbsCopilotLab.slnx
dotnet test CbsCopilotLab.slnx --no-build
dotnet run --project tools\Cbs.Mcp.SmokeClient --no-build -- `
  src\Cbs.Mcp.Server\bin\Debug\net10.0\Cbs.Mcp.Server.dll `
  samples\CBS
```

The included tests cover path inspection, streaming field extraction, successful-session suppression, and current-failure ranking.

5.3 Manual MCP connectivity test

1. Build the solution.
2. Run the smoke client.
3. Confirm all tools are listed.
4. Confirm inspect_cbs returns success: true .

5. Copy the returned `analysisId`.
6. In the Copilot host, ask for a diagnosis and inspect the tool timeline.
7. Confirm `diagnose_cbs` uses that exact ID.

5.4 End-to-end acceptance test

Input:

```
Analyze E:\copilot_mcp_skill_lab\samples\CBS and tell me why the update failed.
```

Expected calls:

```
inspect_cbs(path)
diagnose_cbs(analysisId)
```

Expected semantic result:

- Session `100_1` has an error followed by success and is suppressed.
- Session `101_1` has `0x800F081F`, terminal failure, and rollback.
- The highest-ranked current finding maps to `RepairComponentStore`.
- The answer does not report session `100_1` as the root cause.

5.5 Boundary debugging table

Symptom	Likely layer	How to verify	Logs/debugger	Likely fix
Copilot did not use skill	Skill discovery	Check skill path, frontmatter, trigger wording	Copilot skill/tool timeline, <code>/skills</code> where supported	Correct <code>SKILL.md</code> name/description/location; reload host
Wrong MCP tool selected	Skill/tool metadata	Inspect tool descriptions and prompt ambiguity	MCP client tool-call trace	Make descriptions distinct; strengthen ordering guidance
MCP server did not start	Configuration/processes	Run <code>dotnet <server.dll></code> from configured cwd	Host MCP logs, stderr, Windows process error	Fix command, DLL path, runtime, cwd, or permissions
stdio protocol corruption	MCP interface/logging	Look for non-JSON output on stdout	Capture process stdout/stderr separately	Remove <code>Console.WriteLine</code> ; route diagnostics to stderr
Tool absent from <code>tools/list</code>	Tool discovery	Run smoke client and list names	Server startup logs, debugger in assembly scan	Add <code>[McpServerToolType]</code> , <code>[McpServerTool]</code> , public method, assembly registration

Symptom	Likely layer	How to verify	Logs/debugger	Likely fix
Incorrect parameters	MCP binding/contracts	Compare generated schema with requested JSON	MCP client call payload	Improve descriptions/types; preserve nullable/required semantics
Analysis ID not found	State	Confirm same server process and ID	Repository/application logs	Re-run inspect; increase expiry or add durable storage
CBS path rejected	Inspector/security	Canonicalize path; inspect attributes and access	Inspector debugger, stderr	Supply existing readable directory; avoid reparse root
Parser finds no records	Parser/input	Confirm parseable files and recognizable CBS lines	Parser counters and sample line debugger	Expand supported formats without weakening evidence retention
Diagnosis has no finding	Diagnosis logic/data	Inspect records, candidates, terminal states, suppressed count	Engine debugger, report JSON	Verify session patterns; distinguish valid no-current-failure from parser gap
Serialization failure	Contracts/MCP SDK	Serialize DTO in isolated test	SDK/server exception on stderr	Use public serializable properties and supported types
Tool timeout	Any long-running layer	Check file sizes, cancellation, lock contention	Timings at tool/service/parser boundaries	Bound input, stream, honor cancellation, tune host timeout

5.6 Debugging order

Debug from the outside inward:

```
Prompt
-> skill activation
-> selected tool and arguments
-> MCP process launch
-> initialize/tools/list
-> CbsTools breakpoint
-> LocalCbsToolService breakpoint
-> inspector/parser/engine breakpoint
-> serialized ToolResult
-> Copilot interpretation
```

This order prevents spending time in the parser when the server was never launched or the skill selected the wrong tool.

5.7 Failure injection exercises

1. Pass a missing directory and verify `PATH_NOT_FOUND`.
2. Remove read permission and verify `ACCESS_DENIED`.
3. Restart the server after inspection and verify `ANALYSIS_NOT_FOUND`.
4. Add an old failed session followed by success and verify suppression.

5. Add a current failed session and verify it outranks historical evidence.
6. Cancel a large parse and verify `CANCELLED`.
7. Temporarily write to stdout in the server, observe protocol failure, then restore stderr-only logging.

6. Security and Observability

6.1 Security controls

Risk	Control in this lab	Production extension
Path confusion	<code>Path.GetFullPath</code> and existence check	Allowlisted roots; explicit user-selected paths
Reparsing traversal	Reject root reparsing point and skip child reparsing directories	Resolve final handles and enforce volume/root policy
Unexpected files	CBS name classification and parseable extension filter	Content sniffing and archive validation
Large input	Artifact-count and aggregate-byte limits	Per-file limits, quotas, and streaming archive extraction
Memory exhaustion	Sequential <code>StreamReader</code> , no <code>ReadAllLines</code> , and a relevant-record cap	Bounded channels and incremental aggregation
Long-running work	<code>CancellationToken</code> propagated through I/O and locks	Explicit per-tool deadlines and host quotas
Concurrent duplicate work	Per-analysis <code>SemaphoreSlim</code>	Distributed lock if repository is distributed
Sensitive log content	Local processing and local artifacts	Redaction policy, encryption, retention, access auditing
Remote transmission	None in core lab	Explicit consent, TLS, tenant boundary, data classification
Excessive tool permission	Read-only diagnosis tools	Separate server/identity for state-changing remediation
Arbitrary command execution	No command parameter or shell invocation	Deterministic action registry; signed packages
Error disclosure	Safe <code>ToolError</code> messages; details logged on stderr	Correlation IDs and operator-only diagnostics

6.2 Trust boundaries

```
Untrusted user path
-> validated inspector
-> local read-only file access

Untrusted log text
-> parser treats it as data
-> structured records
-> escaped JSON serialization

Copilot-selected tool arguments
-> application validation
-> no implicit trust because an LLM produced them
```

Tool descriptions improve selection but never replace validation or authorization.

6.3 Consent

Diagnosis is read-only but can expose sensitive local logs. The host should make tool execution visible and obey its approval model.

Remediation must be a separate, explicit, state-changing phase. Require:

- A diagnosed and evidence-backed action
- A deterministic allowlisted `actionId`
- User consent
- Authorization
- Package integrity verification
- Execution audit
- Post-action verification

6.4 Observability path

```
Copilot skill/tool decision
-> MCP client lifecycle and tool-call logs
-> transport/process diagnostics
-> MCP server stderr
-> application service operation logs
-> parser counts/timings
-> diagnosis suppression/ranking diagnostics
-> artifact paths and correlation by analysisId
```

Use `analysisId` as the correlation key across application logs and artifacts. A production implementation should also attach a per-call correlation ID.

6.5 What to log

Layer	Log
MCP process	Startup, shutdown, protocol exceptions on stderr
Tool boundary	Tool name, analysis ID, elapsed time, success/error code
Inspector	Canonical root, artifact count, skipped reparse points, total bytes
Parser	Files parsed, records emitted, candidates emitted, cancellation
Diagnosis	Session count, candidate count, suppression reasons, finding count
Repository	Add, hit, miss, expiry; never dump full sensitive state by default
Artifact writer	Destination paths and write outcome

Avoid logging full CBS lines by default because they may contain machine, package, or environment details. Exact evidence belongs in the explicitly requested result/report with appropriate local access controls.

6.6 stdout versus stderr

For stdio MCP:

```
stdin  = MCP requests
stdout = MCP responses and notifications
stderr = diagnostics
```

Mixing application logs into stdout can make a valid JSON-RPC stream unparsable. The server configures `AddConsole` with `LogToStandardErrorThreshold = Trace` , which routes all configured console logs to stderr.

7. Optional Remediation Extension

This phase is intentionally separate from the read-only diagnosis lab.

7.1 Extended architecture

```
CBS Diagnosis
-> Recommended remediation
-> deterministic actionId
-> Generate handoff
-> GetHelp
-> Validate handoff and consent
-> DiagSvc
-> GetHelp Server
-> Select DAF by actionId
-> Download signed DAF
-> Verify publisher/hash/policy
-> Execute remediation
-> Return structured execution result
-> Copilot
-> rerun inspect_cbs and diagnose_cbs
-> compare before/after state
```

The diagnosis MCP server should not silently become a general command runner. Prefer a separate remediation capability with stronger authorization and audit.

7.2 Handoff contract

```
{
  "analysisId": "A-123",
  "scenario": "CBS_UPDATE_FAILURE",
  "actionId": "RepairComponentStore",
  "evidence": {
    "hresult": "0x800F081F"
  }
}
```

The C# contract is `RemediationHandoff`.

Why `actionId` is deterministic

- It maps to an allowlisted, versioned remediation definition.
- Policy can approve or deny it without interpreting natural language.
- Telemetry can aggregate success and rollback rates.
- The server can prevent parameter injection and arbitrary command execution.
- Repeated diagnoses produce the same action intent for the same condition.

Why evidence accompanies remediation

- The executor can validate that the action matches the diagnosed condition.
- Auditors can reconstruct why a state-changing operation was proposed.
- Policy can require specific HRESULTS, confidence, or evidence classes.
- Post-remediation verification can compare the original finding.

Why arbitrary commands are excluded

A handoff such as `{ "command": "powershell ..." }` transfers unbounded authority from model-generated text to an executor. It is difficult to authorize, validate, sign, audit, or make idempotent. `actionId` should resolve server-side to a reviewed implementation.

7.3 Suggested remediation contracts

```
RemediationRequest
  analysisId
  scenario
  actionId
  evidence
  userConsentToken

RemediationResult
  executionId
  actionId
  status
  startedAt
  completedAt
  rebootRequired
  safeSummary
  verificationRequired
```

7.4 Verification loop

Never equate “remediation command exited successfully” with “Windows Update is fixed.”

```
Execution result
-> new inspect_cbs
-> new analysisId
-> new diagnose_cbs
-> compare current finding/session/terminal state
-> tell user whether the original condition is gone
```

The verification analysis must be new because the old repository state and artifacts describe the pre-remediation filesystem.

8. Hands-On Exercises and Final Mental Model

Exercise 1 - Architecture before implementation

1. Read the HLD.
2. For each box, state its owned decision.
3. Identify forbidden responsibilities:
 - Copilot must not parse CBS.
 - `SKILL.md` must not contain the parser.
 - `CbsTools` must not enumerate files.
 - The parser must not decide MCP errors.
4. Draw the inspect and diagnose call chains from memory.

Exercise 2 - Contracts first

Inspect `Cbs.Mcp.Contracts` and answer:

- Why is `EvidenceReference` separate from `FailureFinding`?
- Why can `Timestamp` and `HResult` be null?
- Why is `ToolResult<T>` generic?
- Which errors are expected operational outcomes rather than exceptions?

Add a serialization test for one success and one failure result.

Exercise 3 - Minimal MCP server

Temporarily register only a simple test tool, build the server, and use the smoke client to list it. Then restore the four CBS tools.

Observe:

- Attributes create tool metadata.
- The MCP client does not call C# methods directly.
- The MCP server performs parameter binding.
- stdout must remain protocol-only.

Exercise 4 - Skill behavior

Use these prompts:

```
Analyze E:\copilot_mcp_skill_lab\samples\CBS and determine why the update failed.
```

```
Show the exact CBS evidence and line numbers for the primary finding.
```

```
Trace the failure from initiation through rollback.
```

Expected tool sequences:

```
inspect_cbs -> diagnose_cbs  
inspect_cbs -> diagnose_cbs -> show_cbs_evidence  
inspect_cbs -> diagnose_cbs -> trace_cbs_failure
```

Exercise 5 - Parser

Add CBS lines with:

- No timestamp
- Lowercase HRESULT
- No package
- Warning severity
- Terminal success
- Terminal failure
- Rollback

Assert the parser retains exact evidence and source line numbers. Do not modify the parser to “fix” evidence text.

Exercise 6 - Diagnosis semantics

Create two sessions:

```
Old session: error -> success  
New session: error -> failure -> rollback
```

Expected result:

- Old error suppressed.
- New error retained.
- Latest completed session points to the new session.
- `SuppressedCandidateCount` is nonzero.

This exercise demonstrates why `grep` for “Error” is not a diagnosis engine.

Exercise 7 - Error paths

Input	Expected result
Null path	INVALID_PATH
Missing directory	PATH_NOT_FOUND
Expired/unknown analysis ID	ANALYSIS_NOT_FOUND
Cancellation	CANCELLED
Unexpected parser failure	ANALYSIS_FAILED plus stderr details

Verify Copilot explains the error and does not invent a root cause.

Exercise 8 - Expected tool responses

Inspection shape:

```
{
  "success": true,
  "data": {
    "analysisId": "A-...",
    "inputPath": "E:\\copilot_mcp_skill_lab\\samples\\CBS",
    "workspaceDirectory": "C:\\Users\\...\\CbsMcpLab\\analyses\\A-...",
    "logDomain": "Windows Component Based Servicing",
    "servicingEvidenceFound": true,
    "totalBytes": 638,
    "artifacts": [],
    "warnings": []
  },
  "error": null
}
```

Diagnosis shape:

```

{
  "success": true,
  "data": {
    "analysisId": "A-...",
    "summary": "The highest-ranked current failure is CBS servicing failure 0X800F081F.",
    "latestCompletedSession": "101_1",
    "lastSuccessfulServicingPoint": "2026-09-25T10:00:05+00:00",
    "findings": [
      {
        "findingId": "F-...",
        "hResult": "0X800F081F",
        "session": "101_1",
        "package": "Package_for_KB5000002",
        "rank": 1,
        "actionId": "RepairComponentStore"
      }
    ],
    "suppressedCandidateCount": 1
  },
  "error": null
}

```

Exact IDs, timestamps, byte counts, and generated paths vary.

Exercise 9 - Extend safely

Possible extensions:

- Parse compressed `CbsPersist*.cab` through a bounded archive abstraction.
- Add durable encrypted repository storage.
- Add HRESULT family knowledge as data, not a giant tool handler.
- Add schema validation for contracts.
- Add a remote HTTP MCP transport with authentication and strict host validation.
- Add a separate remediation MCP server that accepts only allowlisted `actionId` values.

Complete control-flow summary

1. User expresses troubleshooting intent.
2. Copilot reasons that CBS skill applies.
3. SKILL.md supplies ordered workflow and safety rules.
4. MCP client discovers tools from the configured server.
5. `inspect_cbs` crosses the protocol boundary.
6. `CbsTools` forwards the typed request.
7. `LocalCbsToolService` coordinates inspection, workspace, and state.
8. `diagnose_cbs` retrieves state and invokes parser plus diagnosis engine.
9. Domain logic suppresses obsolete evidence and ranks current findings.
10. Contracts carry the result through MCP.
11. Copilot turns the structured result into a user-facing explanation.
12. Optional remediation uses a separate deterministic, consented handoff.

Final mental model

```
COPILOT
  decides what needs to happen

SKILL.md
  tells Copilot the workflow to follow

MCP
  provides the protocol boundary

MCP TOOL
  exposes an operation

APPLICATION SERVICE
  coordinates the operation

DOMAIN ENGINE
  performs the real analysis

CONTRACT
  carries structured information

COPILOT
  converts structured results into a user-facing answer
```

Core principle:

```
Copilot owns reasoning and orchestration.
SKILL.md owns workflow instructions.
MCP owns capability exposure.
Application services own use-case orchestration.
Domain components own CBS intelligence.
Contracts keep every boundary explicit.
```

9. Exact Source Map

Use this map while tracing the implementation in a debugger. Line numbers refer to the completed lab source.

MCP bootstrap and discovery

Concern	Source
Create Generic Host	<code>src\Cbs.Mcp.Server\Program.cs:10</code>
Route console diagnostics to stderr	<code>src\Cbs.Mcp.Server\Program.cs:12-16</code>
Register domain/application dependencies	<code>src\Cbs.Mcp.Server\Program.cs:18-24</code>
Register MCP server	<code>src\Cbs.Mcp.Server\Program.cs:27</code>
Select stdio transport	<code>src\Cbs.Mcp.Server\Program.cs:28</code>
Discover attributed tools	<code>src\Cbs.Mcp.Server\Program.cs:29</code>

Skill and client configuration

Concern	Source
Skill objective	<code>.github\skills\cbs-troubleshooter\SKILL.md:8-10</code>
Inspect-before-diagnose rule	<code>.github\skills\cbs-troubleshooter\SKILL.md:14-28</code>
Evidence escalation	<code>.github\skills\cbs-troubleshooter\SKILL.md:31-36</code>
Failure trace escalation	<code>.github\skills\cbs-troubleshooter\SKILL.md:38-43</code>
Safety rules	<code>.github\skills\cbs-troubleshooter\SKILL.md:57-66</code>
Tool mapping	<code>.github\skills\cbs-troubleshooter\SKILL.md:68-75</code>
VS Code stdio server entry	<code>.vscode\mcp.json:2-12</code>
Portable concrete-path example	<code>config\mcp.generic.json:2-12</code>

MCP tool layer

Tool	Source
Tool class and DI dependency	src\Cbs.Mcp.Server\Tools\CbsTools.cs:8-9
inspect_cbs	src\Cbs.Mcp.Server\Tools\CbsTools.cs:11-20
diagnose_cbs	src\Cbs.Mcp.Server\Tools\CbsTools.cs:22-30
show_cbs_evidence	src\Cbs.Mcp.Server\Tools\CbsTools.cs:32-42
trace_cbs_failure	src\Cbs.Mcp.Server\Tools\CbsTools.cs:44-54

Application service

Operation	Interface	Implementation
Inspect	src\Cbs.Mcp.Server\Services\ILocalCbsToolService.cs:7-9	src\Cbs.Mcp.Server\Services\LocalCbsToolService.cs:18-69
Diagnose	src\Cbs.Mcp.Server\Services\ILocalCbsToolService.cs:11-13	src\Cbs.Mcp.Server\Services\LocalCbsToolService.cs:71-124
Evidence	src\Cbs.Mcp.Server\Services\ILocalCbsToolService.cs:15-18	src\Cbs.Mcp.Server\Services\LocalCbsToolService.cs:126-152
Trace	src\Cbs.Mcp.Server\Services\ILocalCbsToolService.cs:20-23	src\Cbs.Mcp.Server\Services\LocalCbsToolService.cs:154-186
Cached diagnosis helper	n/a	src\Cbs.Mcp.Server\Services\LocalCbsToolService.cs:188-199
Analysis validation/error mapping	n/a	src\Cbs.Mcp.Server\Services\LocalCbsToolService.cs:213-242

Domain engine

Concern	Source
Safe path inspection entry	src\Cbs.Mcp.Server\Analysis\CbsPathInspector.cs:16-114
CBS artifact classification	src\Cbs.Mcp.Server\Analysis\CbsPathInspector.cs:116-137
Parse workflow and record cap	src\Cbs.Mcp.Server\Analysis\CbsLogParser.cs:10-46

Concern	Source
Streaming file reader	src\Cbs.Mcp.Server\Analysis\CbsLogParser.cs:50-76
Field extraction	src\Cbs.Mcp.Server\Analysis\CbsLogParser.cs:78-111
Candidate scoring	src\Cbs.Mcp.Server\Analysis\CbsLogParser.cs:144-164
Terminal-state detection	src\Cbs.Mcp.Server\Analysis\CbsLogParser.cs:166-188
Regex definitions	src\Cbs.Mcp.Server\Analysis\CbsLogParser.cs:190-206
Diagnosis workflow	src\Cbs.Mcp.Server\Analysis\CbsDiagnosisEngine.cs:7-82
Same-session success suppression	src\Cbs.Mcp.Server\Analysis\CbsDiagnosisEngine.cs:84-98
Historical failure suppression	src\Cbs.Mcp.Server\Analysis\CbsDiagnosisEngine.cs:100-115
Finding construction/ranking data	src\Cbs.Mcp.Server\Analysis\CbsDiagnosisEngine.cs:117-153
HRESULT-to-action mapping	src\Cbs.Mcp.Server\Analysis\CbsDiagnosisEngine.cs:155-167
Rollback consequence suppression	src\Cbs.Mcp.Server\Analysis\CbsDiagnosisEngine.cs:169-181
Failure trace construction	src\Cbs.Mcp.Server\Analysis\CbsFailureTracer.cs:7-77

State and artifacts

Concern	Source
Analysis cache object and lock	src\Cbs.Mcp.Server\State\AnalysisState.cs:5-34
Repository add	src\Cbs.Mcp.Server\State\AnalysisRepository.cs:12-21
Repository lookup	src\Cbs.Mcp.Server\State\AnalysisRepository.cs:23-34
Expiry cleanup	src\Cbs.Mcp.Server\State\AnalysisRepository.cs:36-44
Workspace creation	src\Cbs.Mcp.Server\Infrastructure\CbsAnalysisArtifactWriter.cs:23-28
JSON/text report writing	src\Cbs.Mcp.Server\Infrastructure\CbsAnalysisArtifactWriter.cs:30-62

Contracts

Contract	Source
ToolError , ToolResult<T>	src\Cbs.Mcp.Contracts\ToolResult.cs:3-24

Contract	Source
CbsInspectionResult	src\Cbs.Mcp.Contracts\CbsInspectionResult.cs:3-11
ParsedCbsRecord , FailureCandidate	src\Cbs.Mcp.Contracts\ParsedCbsRecord.cs:3-34
FailureFinding , evidence	src\Cbs.Mcp.Contracts\FailureFinding.cs:3-20
Diagnosis, evidence, trace results	src\Cbs.Mcp.Contracts\CbsDiagnosisResult.cs:3-30
Remediation handoff	src\Cbs.Mcp.Contracts\RemediationHandoff.cs:3-9

Tests and live protocol validation

Validation	Source
Path inspector behavior	tests\Cbs.Mcp.Tests\CbsPathInspectorTests.cs:11-40
Streaming parser extraction	tests\Cbs.Mcp.Tests\CbsLogParserTests.cs:12-43
Successful-session suppression	tests\Cbs.Mcp.Tests\CbsDiagnosisEngineTests.cs:9-34
Current failure ranking/deduplication	tests\Cbs.Mcp.Tests\CbsDiagnosisEngineTests.cs:37-67
MCP initialization and tools/list	tools\Cbs.Mcp.SmokeClient\Program.cs:20-46
inspect_cbs protocol call	tools\Cbs.Mcp.SmokeClient\Program.cs:48-70
diagnose_cbs protocol call	tools\Cbs.Mcp.SmokeClient\Program.cs:72-94
Evidence and trace protocol calls	tools\Cbs.Mcp.SmokeClient\Program.cs:96-116